

## Assignment no - 2

### Week 2: Numpy & Pandas for Data Analytics

```
import numpy as np

import pandas as pd

# Create NumPy array containing at least 10 numbers
numbers = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

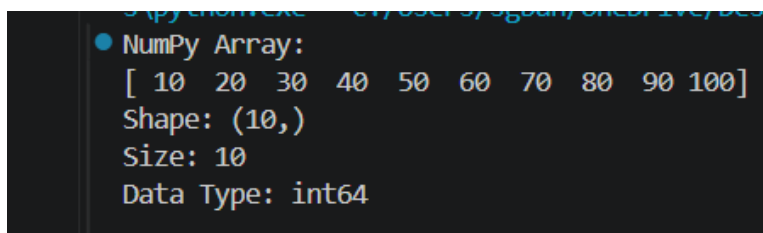
print("NumPy Array:")
print(numbers)

# Display shape
print("Shape:", numbers.shape)

# Display size
print("Size:", numbers.size)

# Display data type
print("Data Type:", numbers.dtype)
```

#### Output:

A screenshot of a Jupyter Notebook cell showing the output of the code. The output is as follows:

```
NumPy Array:
[ 10  20  30  40  50  60  70  80  90 100]
Shape: (10,)
Size: 10
Data Type: int64
```

```
# One-dimensional array
one_d = np.array([1, 2, 3, 4, 5, 6])

print("\nOne-Dimensional Array:")
print(one_d)
```

```
# Two-dimensional array

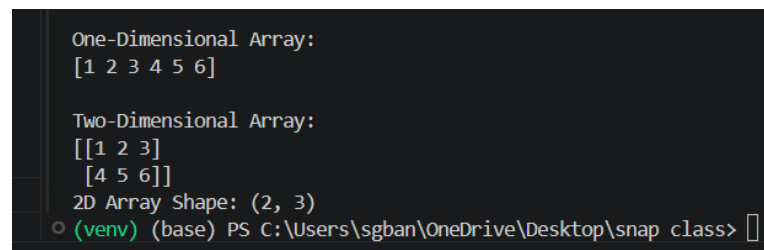
two_d = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print("\nTwo-Dimensional Array:")

print(two_d)

print("2D Array Shape:", two_d.shape)
```

### Output:



```
One-Dimensional Array:
[1 2 3 4 5 6]

Two-Dimensional Array:
[[1 2 3]
 [4 5 6]]
2D Array Shape: (2, 3)
(venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>
```

### # TASK 2: NumPy Indexing, Slicing & Reshaping

```
print("\n===== TASK 2: Indexing, Slicing & Reshaping =====")
```

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
```

#### # Indexing

```
print("First element:", arr[0])

print("Fifth element:", arr[4])

print("Last element:", arr[-1])
```

#### # Slicing

```
print("\nFirst five elements:")

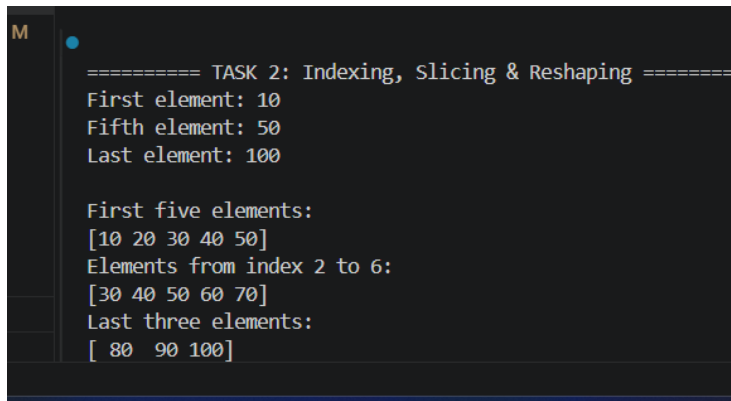
print(arr[:5])

print("Elements from index 2 to 6:")

print(arr[2:7])
```

```
print("Last three elements:")  
print(arr[-3:])
```

**Output:**



The screenshot shows a Jupyter Notebook terminal with the following output:

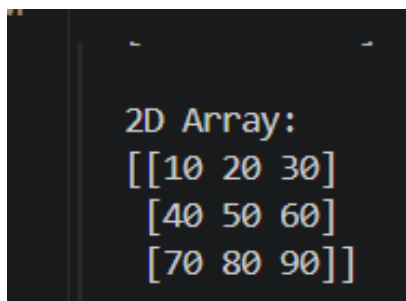
```
===== TASK 2: Indexing, Slicing & Reshaping =====  
First element: 10  
Fifth element: 50  
Last element: 100  
  
First five elements:  
[10 20 30 40 50]  
Elements from index 2 to 6:  
[30 40 50 60 70]  
Last three elements:  
[ 80 90 100]
```

# Create 2D array

```
matrix = np.array([  
    [10, 20, 30],  
    [40, 50, 60],  
    [70, 80, 90]  
])
```

```
print("\n2D Array:")  
print(matrix)
```

**Output:**



The screenshot shows a Jupyter Notebook terminal with the following output:

```
2D Array:  
[[10 20 30]  
 [40 50 60]  
 [70 80 90]]
```

# Access rows

```
print("\nFirst Row:")  
print(matrix[0])
```

```
print("Second Row:")
```

```
print(matrix[1])
```

```
# Access columns
```

```
print("First Column:")
```

```
print(matrix[:, 0])
```

```
print("Second Column:")
```

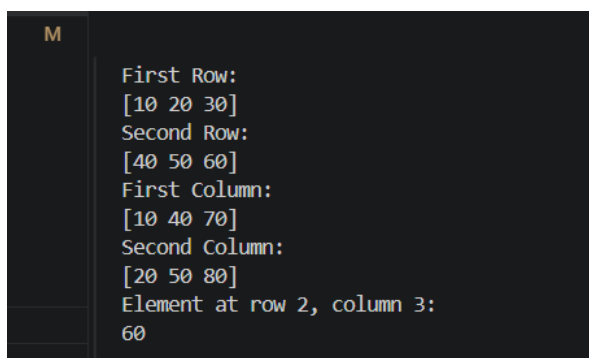
```
print(matrix[:, 1])
```

```
# Access specific element
```

```
print("Element at row 2, column 3:")
```

```
print(matrix[1, 2])
```

**Output:**



```
M
First Row:
[10 20 30]
Second Row:
[40 50 60]
First Column:
[10 40 70]
Second Column:
[20 50 80]
Element at row 2, column 3:
60
```

```
# Reshaping
```

```
original = np.array([
```

```
    1, 2, 3, 4,
```

```
    5, 6, 7, 8,
```

```
    9, 10, 11, 12
```

```
])
```

```
reshaped = original.reshape(3, 4)
```

```
print("\nOriginal Array:")
```

```
print(original)
```

```
print("\nReshaped Array:")
```

```
print(reshaped)
```

**Output:**

```
Original Array:
[ 1  2  3  4  5  6  7  8  9 10 11 12]

Reshaped Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
○ (venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class> |
```

```
# TASK 3: NumPy Mathematical & Statistical Operations
```

```
print("\n===== TASK 3: Mathematical & Statistical Operations =====")
```

```
a = np.array([10, 20, 30, 40, 50])
```

```
b = np.array([5, 10, 15, 20, 25])
```

```
print("Array A:", a)
```

```
print("Array B:", b)
```

```
# Mathematical operations
```

```
print("\nAddition:")
```

```
print(a + b)
```

```
print("\nSubtraction:")
```

```
print(a - b)
```

**Output:**

```
s\python.exe "c:/Users/sgban/OneDrive/Desktop/snap class/src/screens/task.py"
===== TASK 3: Mathematical & Statistical Operations =====
Array A: [10 20 30 40 50]
Array B: [ 5 10 15 20 25]

Addition:
[15 30 45 60 75]

Subtraction:
[ 5 10 15 20 25]
```

```
print("\nMultiplication:")  
print(a * b)
```

```
print("\nDivision:")  
print(a / b)
```

**Output:**

```
Multiplication:  
[ 50 200 450 800 1250]  
  
Division:  
[2. 2. 2. 2. 2.]
```

```
# Statistical dataset  
dataset = np.array([10, 20, 30, 40, 50, 60, 70])
```

```
print("\nStatistical Operations:")
```

```
print("Mean:", np.mean(dataset))
```

```
print("Median:", np.median(dataset))
```

```
print("Minimum:", np.min(dataset))
```

```
print("Maximum:", np.max(dataset))
```

```
print("Standard Deviation:", np.std(dataset))
```

```
print("Sum:", np.sum(dataset))
```

**Output:**

```
Statistical Operations:
Mean: 40.0
Median: 40.0
Minimum: 10
Maximum: 70
Standard Deviation: 20.0
Sum: 280
o (venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>
```

```
print("\n===== TASK 4: Pandas Series & DataFrame =====")
```

```
# Create Pandas Series
```

```
marks_series = pd.Series(
    [85, 78, 92, 88, 76],
    index=["Amit", "Sneha", "Rahul", "Priya", "Neha"]
)
```

```
print("Pandas Series:")
```

```
print(marks_series)
```

**Output:**

```
===== TASK 4: Pandas Series & DataFrame =====
Pandas Series:
Amit      85
Sneha     78
Rahul     92
Priya     88
Neha      76
dtype: int64
```

```
# Create DataFrame
```

```
students = pd.DataFrame({
    "Name": ["Amit", "Sneha", "Rahul", "Priya", "Neha"],
    "Age": [20, 21, 20, 22, 21],
    "Marks": [85, 78, 92, 88, 76],
    "Department": [
        "CSE",
        "Data Science",
```

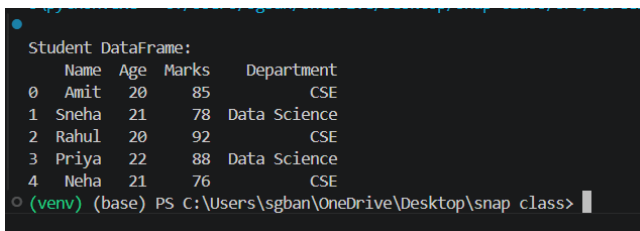
```

        "CSE",
        "Data Science",
        "CSE"
    ]
})

print("\nStudent DataFrame:")
print(students)

```

### Output:



```

Student DataFrame:
   Name  Age  Marks Department
0  Amit   20    85         CSE
1  Sneha  21    78  Data Science
2  Rahul  20    92         CSE
3  Priya  22    88  Data Science
4  Neha   21    76         CSE
(venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>

```

### # Column names

```

print("\nColumn Names:")
print(students.columns)

```

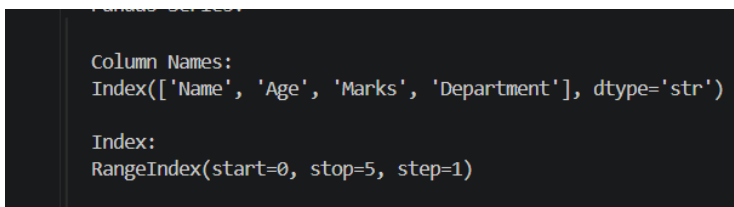
### # Index

```

print("\nIndex:")
print(students.index)

```

### Output:



```

Column Names:
Index(['Name', 'Age', 'Marks', 'Department'], dtype='str')

Index:
RangeIndex(start=0, stop=5, step=1)

```

### # Add new column

```

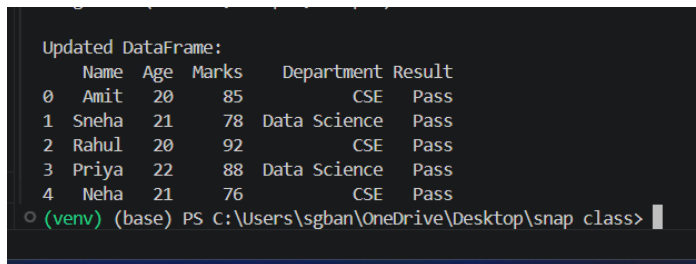
students["Result"] = np.where(
    students["Marks"] >= 40,
    "Pass",

```

```
"Fail"  
)
```

```
print("\nUpdated DataFrame:")  
print(students)
```

### Output:



The screenshot shows a terminal window with a dark background. The output displays the 'Updated DataFrame' with columns: Name, Age, Marks, Department, and Result. The data is as follows:

|   | Name  | Age | Marks | Department   | Result |
|---|-------|-----|-------|--------------|--------|
| 0 | Amit  | 20  | 85    | CSE          | Pass   |
| 1 | Sneha | 21  | 78    | Data Science | Pass   |
| 2 | Rahul | 20  | 92    | CSE          | Pass   |
| 3 | Priya | 22  | 88    | Data Science | Pass   |
| 4 | Neha  | 21  | 76    | CSE          | Pass   |

The terminal prompt at the bottom is: `(venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>`

```
print("\n===== TASK 5: Reading & Inspecting Data =====")
```

```
# Create a sample dataset
```

```
sales_data = pd.DataFrame({  
    "Order_ID": [101, 102, 103, 104, 105,  
                106, 107, 108, 109, 110],
```

```
    "Product": [  
        "Laptop",  
        "Mouse",  
        "Keyboard",  
        "Monitor",  
        "Laptop",  
        "Mouse",  
        "Headphones",  
        "Monitor",  
        "Keyboard",  
        "Laptop"
```

```
],
```

```
"Category": [  
  "Electronics",  
  "Accessories",  
  "Accessories",  
  "Electronics",  
  "Electronics",  
  "Accessories",  
  "Accessories",  
  "Electronics",  
  "Accessories",  
  "Electronics"  
],
```

```
"Region": [  
  "West",  
  "West",  
  "South",  
  "North",  
  "South",  
  "North",  
  "West",  
  "South",  
  "North",  
  "West"  
],
```

```
"Sales": [  
  65000,  
  1200,  
  2500,  
  18000,
```

```

        62000,

        1500,

        3500,

        17000,

        2200,

        68000

    ],

    "Quantity": [

        2, 5, 4, 2, 2,

        6, 3, 2, 4, 2

    ]

})

```

```
# Save dataset as CSV
```

```
sales_data.to_csv("sales_data.csv", index=False)
```

```
# Read CSV using Pandas
```

```
df = pd.read_csv("sales_data.csv")
```

```
print("First 5 Rows:")
```

```
print(df.head())
```

**Output:**

```

===== TASK 5: Reading & Inspecting Data =====
First 5 Rows:
   Order_ID  Product  Category Region  Sales  Quantity
0       101   Laptop  Electronics  West   65000         2
1       102    Mouse  Accessories  West    1200         5
2       103  Keyboard  Accessories  South    2500         4
3       104   Monitor  Electronics  North   18000         2
4       105   Laptop  Electronics  South   62000         2

```

```
print("\nLast 5 Rows:")
```

```
print(df.tail())
```

### Output:



U

```
Last 5 Rows:
```

|   | Order_ID | Product    | Category    | Region | Sales | Quantity |
|---|----------|------------|-------------|--------|-------|----------|
| 5 | 106      | Mouse      | Accessories | North  | 1500  | 6        |
| 6 | 107      | Headphones | Accessories | West   | 3500  | 3        |
| 7 | 108      | Monitor    | Electronics | South  | 17000 | 2        |
| 8 | 109      | Keyboard   | Accessories | North  | 2200  | 4        |
| 9 | 110      | Laptop     | Electronics | West   | 68000 | 2        |

# Number of rows and columns

```
print("\nNumber of Rows and Columns:")
```

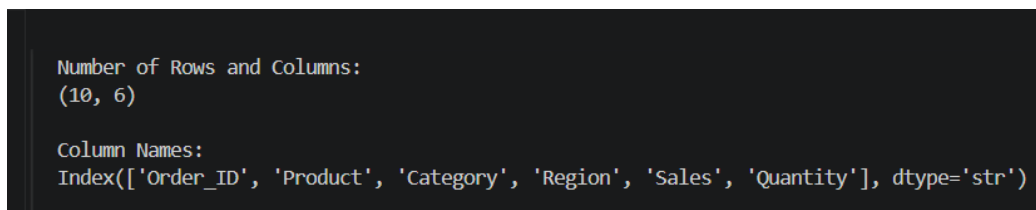
```
print(df.shape)
```

# Column names

```
print("\nColumn Names:")
```

```
print(df.columns)
```

### Output:



```
Number of Rows and Columns:
(10, 6)

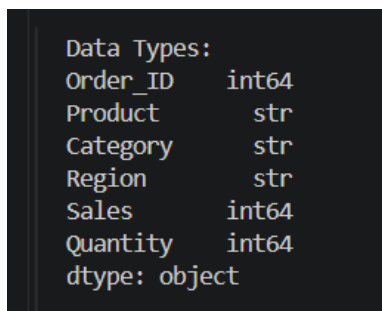
Column Names:
Index(['Order_ID', 'Product', 'Category', 'Region', 'Sales', 'Quantity'], dtype='str')
```

# Data types

```
print("\nData Types:")
```

```
print(df.dtypes)
```

### Output:



```
Data Types:
Order_ID    int64
Product      str
Category     str
Region       str
Sales        int64
Quantity     int64
dtype: object
```

# Info

```
print("\nDataset Information:")
```

```
df.info()
```

### Output:

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Order_ID    10 non-null     int64
1   Product     10 non-null     str
2   Category    10 non-null     str
3   Region      10 non-null     str
4   Sales       10 non-null     int64
5   Quantity    10 non-null     int64
dtypes: int64(3), str(3)
memory usage: 836.0 bytes
```

### # Describe

```
print("\nStatistical Description:")
print(df.describe())
```

### Output:

```
Statistical Description:
      Order_ID      Sales  Quantity
count  10.00000    10.000000    10.00000
mean   105.50000   24090.000000    3.20000
std     3.02765   28924.708776    1.47573
min    101.00000   1200.000000    2.00000
25%    103.25000   2275.000000    2.00000
50%    105.50000  10250.000000    2.50000
75%    107.75000   5100.000000    4.00000
max    110.00000  68000.000000    6.00000
```

### # TASK 6: Selecting, Filtering & Sorting Data

```
print("\n===== TASK 6: Selecting, Filtering & Sorting =====")
```

### # Select specific columns

```
print("Product and Sales Columns:")
print(df[["Product", "Sales"]])
```

### Output:

```

===== TASK 6: Selecting, Filtering & Sorting =====
Product and Sales Columns:
  Product  Sales
0   Laptop 65000
1   Mouse  1200
2  Keyboard  2500
3   Monitor 18000
4   Laptop 62000
5   Mouse  1500
6 Headphones  3500
7   Monitor 17000
8   Keyboard  2200
9   Laptop 68000

```

# Select specific rows

```
print("\nFirst Three Rows:")
```

```
print(df.iloc[:3])
```

# Filter records

```
print("\nSales Greater Than 10,000:")
```

```
print(df[df["Sales"] > 10000])
```

# Multiple conditions

```
print("\nElectronics with Sales Greater Than 20,000:")
```

**Output:**

```

First Three Rows:
  Order_ID  Product  Category Region  Sales  Quantity
0      101   Laptop  Electronics  West  65000         2
1      102    Mouse  Accessories  West   1200         5
2      103  Keyboard  Accessories  South   2500         4

Sales Greater Than 10,000:
  Order_ID  Product  Category Region  Sales  Quantity
0      101   Laptop  Electronics  West  65000         2
3      104  Monitor  Electronics  North 18000         2
4      105   Laptop  Electronics  South 62000         2
7      108  Monitor  Electronics  South 17000         2
9      110   Laptop  Electronics  West 68000         2

Electronics with Sales Greater Than 20,000:
  Order_ID  Product  Category Region  Sales  Quantity
0      101   Laptop  Electronics  West  65000         2
4      105   Laptop  Electronics  South 62000         2
9      110   Laptop  Electronics  West 68000         2

```

```
filtered_data = df[
```

```
    (df["Category"] == "Electronics") &
```

```
    (df["Sales"] > 20000)
```

```
]
```

```
print(filtered_data)
```

```
# Ascending order
```

```
print("\nSales in Ascending Order:")
```

```
print(df.sort_values(
```

```
    by="Sales",
```

```
    ascending=True
```

```
))
```

**Output:**

```
Sales in Ascending Order:
```

|   | Order_ID | Product    | Category    | Region | Sales | Quantity |
|---|----------|------------|-------------|--------|-------|----------|
| 1 | 102      | Mouse      | Accessories | West   | 1200  | 5        |
| 5 | 106      | Mouse      | Accessories | North  | 1500  | 6        |
| 8 | 109      | Keyboard   | Accessories | North  | 2200  | 4        |
| 2 | 103      | Keyboard   | Accessories | South  | 2500  | 4        |
| 6 | 107      | Headphones | Accessories | West   | 3500  | 3        |
| 7 | 108      | Monitor    | Electronics | South  | 17000 | 2        |
| 3 | 104      | Monitor    | Electronics | North  | 18000 | 2        |
| 4 | 105      | Laptop     | Electronics | South  | 62000 | 2        |
| 0 | 101      | Laptop     | Electronics | West   | 65000 | 2        |
| 9 | 110      | Laptop     | Electronics | West   | 68000 | 2        |

```
# Descending order
```

```
print("\nSales in Descending Order:")
```

```
print(df.sort_values(
```

```
    by="Sales",
```

```
    ascending=False
```

```
))
```

**Output:**

```
Sales in Descending Order:
```

|   | Order_ID | Product    | Category    | Region | Sales | Quantity |
|---|----------|------------|-------------|--------|-------|----------|
| 9 | 110      | Laptop     | Electronics | West   | 68000 | 2        |
| 0 | 101      | Laptop     | Electronics | West   | 65000 | 2        |
| 4 | 105      | Laptop     | Electronics | South  | 62000 | 2        |
| 3 | 104      | Monitor    | Electronics | North  | 18000 | 2        |
| 7 | 108      | Monitor    | Electronics | South  | 17000 | 2        |
| 6 | 107      | Headphones | Accessories | West   | 3500  | 3        |
| 2 | 103      | Keyboard   | Accessories | South  | 2500  | 4        |
| 8 | 109      | Keyboard   | Accessories | North  | 2200  | 4        |
| 5 | 106      | Mouse      | Accessories | North  | 1500  | 6        |
| 1 | 102      | Mouse      | Accessories | West   | 1200  | 5        |

```
o (venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>
```

```
print("\n===== TASK 7: Handling Missing Values =====")
```

```
# Dataset containing missing values
```

```
missing_data = pd.DataFrame({
```

```
    "Name": [
```

```
        "Amit",
```

```
        "Sneha",
```

```
        "Rahul",
```

```
        "Priya",
```

```
        "Neha"
```

```
    ],
```

```
    "Age": [
```

```
        20,
```

```
        21,
```

```
        np.nan,
```

```
        22,
```

```
        21
```

```
    ],
```

```
    "Marks": [
```

```
        85,
```

```
        np.nan,
```

```
        92,
```

```
        88,
```

```
        np.nan
```

```
    ],
```

```
    "City": [
```

```
        "Pune",
```

```
        "Mumbai",
```

```

        "Kolhapur",
        None,
        "Pune"
    ]
})

print("Dataset Before Handling Missing Values:")
print(missing_data)

```

```

# Identify missing values

print("\nMissing Values using isnull():")
print(missing_data.isnull())

```

### Output:

```

===== TASK 7: Handling Missing Values =====
Dataset Before Handling Missing Values:
   Name  Age  Marks  City
0  Amit  20.0   85.0   Pune
1  Sneha  21.0    NaN  Mumbai
2  Rahul  NaN   92.0  Kolhapur
3  Priya  22.0   88.0    NaN
4  Neha  21.0    NaN   Pune

Missing Values using isnull():
   Name  Age  Marks  City
0  False  False  False  False
1  False  False   True  False
2  False   True  False  False
3  False  False  False   True
4  False  False   True  False

```

```

# Count missing values

print("\nMissing Values in Each Column:")
print(missing_data.isnull().sum())

# Remove rows containing missing values
removed_data = missing_data.dropna()

print("\nAfter Removing Rows with Missing Values:")
print(removed_data)

```

```
# Fill missing values
```

```
filled_data = missing_data.copy()
```

```
filled_data["Age"] = filled_data["Age"].fillna(  
    filled_data["Age"].mean()  
)
```

```
filled_data["Marks"] = filled_data["Marks"].fillna(  
    filled_data["Marks"].mean()  
)
```

```
filled_data["City"] = filled_data["City"].fillna(  
    "Unknown"  
)
```

```
print("\nAfter Filling Missing Values:")
```

```
print(filled_data)
```

**Output:**

```
Missing Values in Each Column:  
Name      0  
Age        1  
Marks      2  
City        1  
dtype: int64  
  
After Removing Rows with Missing Values:  
   Name  Age  Marks  City  
0  Amit  20.0   85.0  Pune  
  
After Filling Missing Values:  
   Name  Age  Marks  City  
0  Amit  20.0  85.000000  Pune  
1  Sneha  21.0  88.333333  Mumbai  
2  Rahul  21.0  92.000000  Kolhapur  
3  Priya  22.0  88.000000  Unknown  
4   Neha  21.0  88.333333   Pune
```

```
print("\nWhy Handling Missing Data is Important:")
```

```
print(  

```

"Handling missing data is important because it improves "  
"data quality, prevents errors, and helps produce reliable "  
"analysis and conclusions."

)

### Output:

```
Why Handling Missing Data is Important:
Handling missing data is important because it improves data quality, prevents errors, and helps produce reliable analysis and conclusions.
(venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>
```

```
print("\n===== TASK 8: Merge, Concatenate, GroupBy & Pivot Table =====")
```

```
# ----- MERGE -----
```

```
employee_data = pd.DataFrame({
    "Employee_ID": [1, 2, 3, 4],
    "Name": ["Amit", "Sneha", "Rahul", "Priya"],
    "Department": [
        "IT",
        "HR",
        "IT",
        "Sales"
    ]
})
```

```
salary_data = pd.DataFrame({
    "Employee_ID": [1, 2, 3, 4],
    "Salary": [
        50000,
        45000,
        55000,
        48000
    ]
})
```

```

merged_data = pd.merge(
    employee_data,
    salary_data,
    on="Employee_ID"
)

print("\nMerged DataFrame:")
print(merged_data)

# ----- CONCATENATE -----

df1 = pd.DataFrame({
    "Name": ["Amit", "Sneha"],
    "Marks": [85, 90]
})

df2 = pd.DataFrame({
    "Name": ["Rahul", "Priya"],
    "Marks": [78, 88]
})

concatenated_data = pd.concat(
    [df1, df2],
    ignore_index=True
)

print("\nConcatenated DataFrame:")
print(concatenated_data)

# ----- GROUPBY -----

print("\nTotal Sales by Category:")

category_sum = df.groupby(
    "Category"

```

```
)["Sales"].sum()
```

```
print(category_sum)
```

```
print("\nAverage Sales by Category:")
```

```
category_mean = df.groupby(
```

```
    "Category"
```

```
)["Sales"].mean()
```

```
print(category_mean)
```

```
print("\nNumber of Orders by Category:")
```

```
category_count = df.groupby(
```

```
    "Category"
```

```
)["Order_ID"].count()
```

```
print(category_count)
```

### Output:

```
===== TASK 8: Merge, Concatenate, GroupBy & Pivot Table =====

Total Sales by Category:
Category
Accessories    10900
Electronics   230000
Name: Sales, dtype: int64

Average Sales by Category:
Category
Accessories     2180.0
Electronics   46000.0
Name: Sales, dtype: float64

Number of Orders by Category:
Category
Accessories      5
Electronics      5
Name: Order_ID, dtype: int64
```

```
print("\nMinimum Sales by Category:")
```

```
print(  
    df.groupby("Category")["Sales"].min()  
)
```

```
print("\nMaximum Sales by Category:")
```

```
print(  
    df.groupby("Category")["Sales"].max()  
)
```

```
# ----- PIVOT TABLE -----
```

```
pivot_table = pd.pivot_table(  
    df,  
    values="Sales",  
    index="Region",  
    columns="Category",  
    aggfunc="sum",  
    fill_value=0  
)
```

```
print("\nPivot Table:")
```

```
print(pivot_table)
```

**Output:**

```

Minimum Sales by Category:
Category
Accessories      1200
Electronics     17000
Name: Sales, dtype: int64

Maximum Sales by Category:
Category
Accessories      3500
Electronics     68000
Name: Sales, dtype: int64

Pivot Table:
Category Accessories Electronics
Region
North          3700          18000
South          2500          79000
West           4700         133000
○ (venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap_class>

```

# TASK 9: Exporting Data

```
print("\n===== TASK 9: Exporting Data =====")
```

# Export final DataFrame to CSV

```
df.to_csv(
    "processed_sales_data.csv",
    index=False
)
```

```
print(
    "Data successfully exported to "
    "'processed_sales_data.csv'"
)
```

# Read exported file to verify

```
verified_data = pd.read_csv(
    "processed_sales_data.csv"
)
```

```
print("\nVerified Exported Data:")
```

```
print(verified_data)
```

```
print("\nExport Verification Successful!")
```

### Output:

```
===== TASK 9: Exporting Data =====
Data successfully exported to 'processed_sales_data.csv'

Verified Exported Data:
  Order_ID  Product  Category  Region  Sales  Quantity
0       101   Laptop  Electronics  West  65000         2
1       102    Mouse  Accessories  West   1200         5
2       103  Keyboard  Accessories  South   2500         4
3       104   Monitor  Electronics  North  18000         2
4       105   Laptop  Electronics  South  62000         2
5       106    Mouse  Accessories  North   1500         6
6       107 Headphones  Accessories  West    3500         3
7       108   Monitor  Electronics  South  17000         2
8       109  Keyboard  Accessories  North    2200         4
9       110   Laptop  Electronics  West   68000         2

Export Verification Successful!
o (venv) (base) PS C:\Users\sehan\OneDrive\Desktop\snap_class>
```

### # TASK 10: Mini Data Analysis Project

```
print("\n===== TASK 10: MINI DATA ANALYSIS PROJECT =====")
```

```
# Load dataset
```

```
analysis_data = pd.read_csv(
    "sales_data.csv"
)
```

```
print("\n1. Dataset:")
```

```
print(analysis_data)
```

```
# ----- DATA INSPECTION -----
```

```
print("\n2. Data Inspection")
```

```
print("Shape:")
```

```
print(analysis_data.shape)
```

```
print("\nColumns:")
```

```
print(analysis_data.columns)
```

```
print("\nData Types:")
```

```
print(analysis_data.dtypes)
```

**Output:**

```
===== TASK 10: MINI DATA ANALYSIS PROJECT =====

1. Dataset:
   Order_ID  Product  Category Region  Sales  Quantity
0      101    Laptop  Electronics West   65000        2
1      102     Mouse  Accessories West   1200        5
2      103  Keyboard  Accessories South   2500        4
3      104    Monitor  Electronics North  18000        2
4      105    Laptop  Electronics South  62000        2
5      106     Mouse  Accessories North   1500        6
6      107 Headphones  Accessories West    3500        3
7      108    Monitor  Electronics South  17000        2
8      109  Keyboard  Accessories North   2200        4
9      110    Laptop  Electronics West   68000        2

2. Data Inspection
Shape:
(10, 6)

Columns:
Index(['Order_ID', 'Product', 'Category', 'Region', 'Sales', 'Quantity'], dtype='str')

Data Types:
Order_ID    int64
Product     str
Category     str
Region       str
Sales        int64
Quantity     int64
dtype: object
```

```
# ----- MISSING VALUES -----
```

```
print("\n3. Missing Values:")
```

```
print(
    analysis_data.isnull().sum()
)
```

```
# ----- SELECTING DATA -----
```

```
print("\n4. Selected Data:")
```

```
print(
```

```

analysis_data[
    ["Product", "Sales"]
]
)
# ----- FILTERING -----
print("\n5. Sales Greater Than 20,000:")

high_sales = analysis_data[
    analysis_data["Sales"] > 20000
]

print(high_sales)

```

### Output:

```

3. Missing Values:
Order_ID    0
Product      0
Category     0
Region       0
Sales        0
Quantity     0
dtype: int64

4. Selected Data:
   Product  Sales
0   Laptop  65000
1   Mouse   1200
2  Keyboard   2500
3   Monitor 18000
4   Laptop  62000
5   Mouse   1500
6 Headphones  3500
7   Monitor 17000
8  Keyboard   2200
9   Laptop  68000

5. Sales Greater Than 20,000:
   Order_ID Product  Category Region  Sales  Quantity
0        101  Laptop  Electronics  West  65000         2
4        105  Laptop  Electronics  South  62000         2
9        110  Laptop  Electronics  West  68000         2

```

```

# ----- SORTING -----

print("\n6. Sales Sorted in Descending Order:")

sorted_data = analysis_data.sort_values(
    by="Sales",
    ascending=False
)

```

```

)

print(sorted_data)

# ----- GROUPBY ANALYSIS -----

print("\n7. Total Sales by Category:")

category_sales = analysis_data.groupby(
    "Category"
)["Sales"].sum()

print(category_sales)

print("\nAverage Sales by Region:")

region_sales = analysis_data.groupby(
    "Region"
)["Sales"].mean()

print(region_sales)

```

### Output:

```

6. Sales Sorted in Descending Order:
  Order_ID  Product  Category  Region  Sales  Quantity
9       110   Laptop  Electronics  West  68000         2
0       101   Laptop  Electronics  West  65000         2
4       105   Laptop  Electronics  South  62000         2
3       104   Monitor  Electronics  North  18000         2
7       108   Monitor  Electronics  South  17000         2
6       107  Headphones  Accessories  West   3500         3
2       103   Keyboard  Accessories  South   2500         4
8       109   Keyboard  Accessories  North   2200         4
5       106     Mouse  Accessories  North   1500         6
1       102     Mouse  Accessories  West   1200         5

```

```

7. Total Sales by Category:
Category
Accessories    10900
Electronics   230000
Name: Sales, dtype: int64

```

```

Average Sales by Region:
Region
North    7233.333333
South   27166.666667
West   34425.000000
Name: Sales, dtype: float64

```

```
# ----- PIVOT TABLE -----
```

```
print("\n8. Pivot Table:")
```

```
analysis_pivot = pd.pivot_table(
```

```
    analysis_data,
```

```
    values="Sales",
```

```
    index="Region",
```

```
    columns="Category",
```

```
    aggfunc="sum",
```

```
    fill_value=0
```

```
)
```

```
print(analysis_pivot)
```

```
# ----- INSIGHTS -----=
```

```
print("\n9. KEY FINDINGS / INSIGHTS")
```

```
total_sales = analysis_data["Sales"].sum()
```

```
average_sales = analysis_data["Sales"].mean()
```

```
highest_sale = analysis_data["Sales"].max()
```

```
highest_sale_product = analysis_data.loc[
```

```
    analysis_data["Sales"].idxmax(),
```

```
    "Product"
```

```
]
```

```
highest_category = category_sales.idxmax()
```

```
highest_region = (
```

```
    analysis_data
```

```
    .groupby("Region")["Sales"]
```

```
.sum()
.idxmax()
)

print("Total Sales:", total_sales)

print(
    "Average Order Sales:",
    round(average_sales, 2)
)

print(
    "Highest Single Sale:",
    highest_sale
)

print(
    "Product with Highest Sale:",
    highest_sale_product
)

print(
    "Category with Highest Total Sales:",
    highest_category
)

print(
    "Region with Highest Total Sales:",
    highest_region
)

# ----- EXPORT FINAL DATA -----
```

```

analysis_data.to_csv(
    "final_processed_sales_data.csv",
    index=False
)

print(
    "\n10. Final processed dataset "
    "exported successfully."
)

print("\n=====")
print("ALL 10 TASKS COMPLETED SUCCESSFULLY")
print("=====")

```

#### Output:

```

8. Pivot Table:
Category Accessories Electronics
Region
North      3700      18000
South      2500      79000
West       4700     133000

9. KEY FINDINGS / INSIGHTS
Total Sales: 240900
Average Order Sales: 24090.0
Highest Single Sale: 68000
Product with Highest Sale: Laptop
Category with Highest Total Sales: Electronics
Region with Highest Total Sales: West

```

```

10. Final processed dataset exported successfully.

```

```

=====
ALL 10 TASKS COMPLETED SUCCESSFULLY
=====

```

```

○ (venv) (base) PS C:\Users\sgban\OneDrive\Desktop\snap class>

```